BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

Combinatorial Explosion: A table-driven agent requires lookup table mapping for every possible percept sequence. The number of state combinations grows exponentially over time ($|P|^T$). For Chess or grid navigation, storing all possible sequence combinations requires more memory than total atoms in the observable universe.
Effect of Increasing Lifetime: As the agent's lifetime T increases, the table size grows exponentially, rendering memory storage, creation time, and lookup completely intractable.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Mapping

SimpleReflexAgent
code to Condition-Action Rules: In SimpleReflexAgent.sense_and_act:

python
```
if percept.get('food_here'):
    return 'suck'
elif percept.get('wall_ahead'):
    return 'turn_left'
else:
    return 'move_forward'
```

Condition: percept.get('food_here') or percept.get('wall_ahead').
Action: 'suck', 'turn_left', or 'move_forward'.
Mapping: Each if/elif/else branch directly encodes a hardcoded Condition $\rightarrow$ Action mapping that relies strictly on the current percept, ignoring history entirely.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Lack of Percept History: The agent has no memory of past percepts or actions taken previously.
Partial Observability: The agent's sensors only perceive immediate adjacent boundaries (wall_ahead, food_here) without global context (like absolute coordinates

(x, y)
).
Failure Cause: Upon facing a wall or corner, the current percept yields wall_ahead = True. The condition rule triggers 'turn_left'. On the next step, moving forward brings it straight into another boundary wall (e.g. inside a U-shaped barrier). Because the percept remains identical to a previous state, the agent executes the exact same deterministic response indefinitely, causing an inescapable infinite loop.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves)

and the "Sensor Model" (how the agent's actions affect the world).

```
# Transition & Sensor Model Updates
if self.last_action == 'turn_left':
    self.direction = dirs[(dirs.index(self.direction) + 1) % 4]
elif self.last_action == 'move_forward':
    if not self.last_was_wall:
        # Update relative (x, y) coordinates
        self.visited_cells.add((self.x, self.y))
```

Transition Model (How the world works): The code computes how action choices change internal properties over time (e.g., executing move_forward changes position (x, y) based on facing

direction
).
Sensor Model (How actions affect percepts): By updating self.visited_cells and checking left_pos in self.visited_cells, the agent predicts whether turning into a neighboring cell leads to previously visited space.
Decision Rule Evaluation:
python

```
if percept.get('wall_ahead'):
    if left_pos in self.visited_cells:
        action = 'turn_right'  # Avoid repeating loop
    else:
        action = 'turn_left'
```

The internal state (visited_cells) is updated before applying decision rules, enabling the agent to detect repeated states and select alternative actions (turn_right) to escape traps.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

FACULTY OF COMPUTING